# LAPORAN PRAKTIKUM
# ALGORITMA & STRUKTUR DATA
# MODUL 2

## Stack and Queue

**Oleh:**

**Luckya Ari** **NIM : 2510817310015**
**Getsemane Simangunsong**

**PROGRAM STUDI TEKNOLOGI INFORMASI**
**FAKULTAS TEKNIK**
**UNIVERSITAS LAMBUNG MANGKURAT**

**APRIL**
**2026**

# LEMBAR PENGESAHAN
# LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
# MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack and Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Luckya Ari Getsemane Simangunsong
NIM : 2510817310015

| | |
|---|---|
| Menyetujui,<br>Asisten Praktikum | Mengetahui,<br>Dosen Penanggung Jawab Praktikum |
| Muhammad Fauzan Ahsani<br>NIM. 2310817310009 | Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D<br>NIP. 198606132015041001 |

# DAFTAR ISI

LEMBAR PENGESAHAN ..... i

DAFTAR ISI ..... ii

DAFTAR GAMBAR ..... iii

DAFTAR TABEL ..... iv

SOAL 1 Implementasi Stack ..... 1

A. Source Code ..... 1

B. Pembahasan ..... 3

SOAL 2 Si Palui dan Mesin Hitung Aneh ..... 8

A. Source Code ..... 9

B. Output Program ..... 11

C. Pembahasan ..... 11

SOAL 3 Implementasi Queue ..... 13

A. Source Code ..... 13

B. Pembahasan ..... 15

SOAL 4 Si Palui dan Deretan Angka ..... 19

A. Source Code ..... 20

B. Output Program ..... 21

C. Pembahasan ..... 21

TAUTAN GIT ..... 24

# DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 2 ..... 11
Gambar 2. Screenshot Hasil Jawaban Soal 4 ..... 21

# DAFTAR TABEL

Table 1. Source Code stack.h ..... 1
Table 2. Source Code stack.cpp ..... 2
Table 3. Source Code prob1.cpp ..... 9
Table 4. Source Code queue.h ..... 13
Table 5. Source Code queue.cpp ..... 14
Table 6. Source Code prob2.cpp ..... 20

# SOAL 1

# Implementasi Stack

Pada file `stack.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `stack.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis

kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

## A. Source Code

*Table 1. Source Code stack.h*

```
#ifndef STACK_H
#define STACK_H

#define MAX 100

struct Stack {
    int data[MAX];
    int* top;
};

void init(Stack* s);
bool isEmpty(const Stack* s);
bool isFull(const Stack* s);
void push(Stack* s, int value);
void pop(Stack* s);
int peek(const Stack* s);

```

```
#endif
```

*Table 2. Source Code stack.cpp*

```
#include "stack.h"
#include <stdexcept>

void init(Stack* s) {
    s->top = s->data - 1;
}

bool isEmpty(const Stack* s) {
    return s->top < s->data;
}

bool isFull(const Stack* s) {
    return s->top >= s->data + MAX - 1;
}

void push(Stack* s, int value) {
    if (isFull(s))
        throw std::overflow_error("Stack penuh");
    *(++s->top) = value;
}

void pop(Stack* s) {
    if (isEmpty(s))
        throw std::underflow_error("Stack kosong");
    s->top--;
}

int peek(const Stack* s) {
```

```
29      if (isEmpty(s))
30          throw std::underflow_error("Stack kosong");
31      return *(s->top);
32  }
```

## B. Pembahasan

Pembahasan `stack.h`

Pada baris [1], sintaks `#ifndef STACK_H` membuka include guard yang memeriksa apakah makro `STACK_H` belum pernah didefinisikan sebelumnya oleh kompiler. Pada baris [2], `#define STACK_H` mendefinisikan makro tersebut sehingga apabila file `stack.h` di-include lebih dari satu kali dalam proyek yang sama, kompiler tidak akan memproses isinya berulang kali dan terhindar dari error redefinisi.

Pada baris [4], sintaks `#define MAX 100` mendefinisikan konstanta preprocessor `MAX` dengan nilai `100` yang akan digunakan sebagai kapasitas maksimum array di dalam struct. Penggunaan `#define` berarti setiap kemunculan kata `MAX` dalam kode akan diganti langsung oleh kompiler dengan angka `100` sebelum proses kompilasi dimulai.

Pada baris [6–9], didefinisikan `struct Stack` yang merepresentasikan struktur data stack berbasis array. Atribut `data[MAX]` bertipe `int` adalah array berkapasitas 100 elemen yang berfungsi sebagai tempat penyimpanan data stack. Atribut `top` bertipe `int*` adalah pointer yang selalu menunjuk ke alamat memori elemen paling atas stack, digunakan untuk melacak posisi elemen terakhir yang dimasukkan.

Pada baris [11], dideklarasikan prototipe fungsi `init()` yang menerima pointer ke `Stack` dan bertugas menginisialisasi stack ke kondisi awal kosong, biasanya dengan mengatur pointer `top` ke posisi sebelum elemen pertama. Pada baris [12], dideklarasikan prototipe fungsi `isEmpty()` yang menerima pointer `const` ke `Stack` dan mengembalikan `bool` untuk memeriksa apakah stack dalam kondisi kosong. Pada baris [13], dideklarasikan prototipe fungsi `isFull()` yang mengembalikan `bool` untuk memeriksa apakah stack sudah penuh sesuai kapasitas `MAX`. Pada baris [14], dideklarasikan prototipe fungsi `push()` yang menerima pointer ke `Stack` dan nilai `int` untuk mendorong sebuah elemen baru ke atas stack. Pada baris [15], dideklarasikan prototipe fungsi `pop()` yang menerima pointer ke

`Stack` untuk menghapus elemen paling atas dari stack. Pada baris [16], dideklarasikan prototipe fungsi `peek()` yang menerima pointer `const` ke `Stack` dan mengembalikan nilai `int` berupa elemen paling atas stack tanpa menghapusnya. Pada baris [18], `#endif` menutup blok include guard yang dibuka pada baris pertama.

Pembahasan `stack.cpp`

Pada baris [1], sintaks `#include "stack.h"` mengimpor file header stack buatan sendiri yang berisi definisi struct `Stack` beserta deklarasi fungsi-fungsi operasi stack seperti `init()`, `push()`, `pop()`, `peek()`.

Pada baris [2] sintaks `#include <stdexcept>` mengimpor library standar C++ yang menyediakan kelas-kelas exception (pengecualian) bawaan untuk menangani kondisi error saat program berjalan. Library ini digunakan bersama mekanisme `try-catch` untuk menangkap dan melempar error secara terstruktur.

Pada baris [4-5] terdapat Fungsi init

```
void init(Stack* s) {
    s->top = s->data - 1;
```

Fungsi ini menginisialisasi pointer top ke posisi satu slot sebelum awal array data, yaitu data - 1. Posisi ini berfungsi sebagai sentinel yang menandakan bahwa stack berada dalam kondisi kosong sebelum ada elemen yang dimasukkan.

Justifikasi kebenaran: Pilihan nilai awal `top = data - 1` secara langsung mendukung invariant isEmpty, yaitu top < data. Ketika push pertama dilakukan, top dinaikkan menjadi data (slot pertama), sehingga invariant konsisten. Tidak ada kondisi tidak valid pada init karena selalu dapat dieksekusi tanpa prasyarat.

Pada baris [8-9] terdapat Fungsi isEmpty

```
bool isEmpty(const Stack* s) {
    return s->top < s->data;
```

Fungsi ini membandingkan nilai pointer top dengan alamat awal array data. Jika top masih berada di bawah data, berarti tidak ada elemen yang pernah disisipkan atau semua elemen telah dikeluarkan.

Justifikasi kebenaran: Setelah init, top = data - 1 sehingga top < data bernilai true (kosong). Setiap push menaikkan top satu langkah; setiap pop menurunkannya satu langkah. Kondisi top < data hanya bisa terjadi kembali saat semua elemen telah di-pop, sehingga logika ini benar.

Pada baris [12-13] terdapat Fungsi isFull

```
bool isFull(const Stack* s) {
    return s->top >= s->data + MAX - 1;
```

Fungsi ini memeriksa apakah pointer top sudah mencapai elemen terakhir dari array, yaitu posisi data + MAX - 1. Jika ya, tidak ada ruang yang tersisa untuk elemen baru.

Justifikasi kebenaran: Array data memiliki indeks valid dari 0 hingga MAX - 1, yang berarti slot valid terakhir berada di alamat data + MAX - 1. Ketika top mencapai posisi ini, kapasitas penuh telah terpenuhi, sehingga kondisi ini benar secara aritmetika pointer.

Pada baris [16-19] terdapat Fungsi push

```
void push(Stack* s, int value) {
    if (isFull(s))
        throw std::overflow_error("Stack penuh");
    *(++s->top) = value;
```

Fungsi ini memeriksa apakah stack sudah penuh menggunakan isFull. Jika penuh, exception std::overflow_error dilempar. Jika tidak, pointer top dinaikkan terlebih dahulu (pre-increment), kemudian nilai ditulis ke lokasi yang ditunjuk top.

Justifikasi kebenaran: Pre-increment memastikan top selalu menunjuk ke elemen valid yang paling baru dimasukkan. Jika post-increment digunakan, top akan menunjuk ke slot kosong, sehingga peek akan membaca memori yang belum terisi. Urutan increment lalu assign adalah invariant kritis yang dijaga oleh implementasi ini.

Pada baris [22-25] terdapat Fungsi pop

```
void pop(Stack* s) {
    if (isEmpty(s))
        throw std::underflow_error("Stack kosong");
    s->top--;
```

Fungsi ini memeriksa apakah stack kosong. Jika kosong, exception std::underflow_error dilempar. Jika tidak, pointer top diturunkan satu posisi, secara efektif 'menghapus' elemen teratas dari perspektif stack meskipun nilai aslinya masih ada di memori.

Justifikasi kebenaran: Karena push dan pop hanya memanipulasi pointer top, data lama tidak perlu dihapus secara eksplisit. Slot tersebut akan ditimpa oleh push berikutnya. Penurunan top satu posisi memastikan bahwa elemen yang di-push berikutnya menimpa posisi lama, menjaga konsistensi struktur.

Pada baris [28-31]

```
int peek(const Stack* s) {
    if (isEmpty(s))
        throw std::underflow_error("Stack kosong");
    return *(s->top);
```

Fungsi ini memeriksa apakah stack kosong. Jika kosong, exception std::underflow_error dilempar. Jika tidak, fungsi mengembalikan nilai yang ditunjuk oleh top tanpa mengubah posisi top.

Justifikasi kebenaran: Karena push selalu menggunakan pre-increment, top setelah push pasti menunjuk ke elemen yang baru saja dimasukkan. Oleh karena itu, dereferensi *top langsung menghasilkan nilai elemen teratas yang valid.

Analisis Kompleksitas Stack

| **Fungsi** | **Kompleksitas Waktu** | **Kompleksitas Ruang** |
|---|---|---|
| init | O(1) | O(1) |
| isEmpty | O(1) | O(1) |
| isFull | O(1) | O(1) |
| push | O(1) | O(1) |
| pop | O(1) | O(1) |
| peek | O(1) | O(1) |

Seluruh operasi pada stack berjalan dalam kompleksitas waktu O(1) karena setiap operasi hanya melibatkan manipulasi pointer dan akses satu elemen secara langsung, tanpa iterasi. Kompleksitas ruang juga O(1) per operasi karena tidak ada alokasi memori tambahan yang dilakukan; array data bersifat statis dengan ukuran MAX.

# SOAL 2

## Si Palui dan Mesin Hitung Aneh

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

**Batasan**

- $1 \le n \le 100$ • $-1000 \le A_i \le 1000$

Keterangan:

- $n$ adalah jumlah simbol dalam ekspresi • $A_i$ adalah nilai bilangan pada input
- Simbol yang valid hanya: ‣ bilangan bulat

- operator: +, -, *, /

**Format Input**

```
n A1
A2 ... An
```

Keterangan:

- Baris pertama `n` menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol `A` sebanyak `n` buah

**Format Output**

Sebuah bilangan hasil dari kalkulator Si Palui.

**Contoh Kasus**

| Input | Output |
|---|---|
| 3<br>2 3 + | 5 |
| 9<br>5 1 2 + 4 * + 3 - | 14 |
| 5<br>10 2 / 3 * | 15 |

## A. Source Code

*Table 3. Source Code prob1.cpp*

```
#include <iostream>
#include <sstream>
#include <string>
#include "stack.h"

int main() {
    int n;
    std::cin >> n;
    std::cin.ignore();

```

```
    std::string line;
    std::getline(std::cin, line);

    std::istringstream iss(line);
    Stack s;
    init(&s);

    std::string token;
    while (iss >> token) {
        if (token == "+" || token == "-" || token == "*"
|| token == "/") {
            int b = peek(&s); pop(&s);
            int a = peek(&s); pop(&s);

            int result = 0;
            if      (token == "+") result = a + b;
            else if (token == "-") result = a - b;
            else if (token == "*") result = a * b;
            else                   result = a / b;

            push(&s, result);
        } else {
            push(&s, std::stoi(token));
        }
    }

    std::cout << peek(&s) << std::endl;
    return 0;
}
```

## B. Output Program

```
PS C:\KULIAH_TI\SEMESTER 2\Praktikum Algo & Struktur Data\MODUL2\module-2-stack-and-queue-Luckyari21-main> g++ prob1.cpp stack.cpp -o prob
1
PS C:\KULIAH_TI\SEMESTER 2\Praktikum Algo & Struktur Data\MODUL2\module-2-stack-and-queue-Luckyari21-main> ./prob1
3
2 3 +
5
PS C:\KULIAH_TI\SEMESTER 2\Praktikum Algo & Struktur Data\MODUL2\module-2-stack-and-queue-Luckyari21-main> ./prob1
9
5 1 2 + 4 * + 3 -
14
PS C:\KULIAH_TI\SEMESTER 2\Praktikum Algo & Struktur Data\MODUL2\module-2-stack-and-queue-Luckyari21-main> ./prob1
5
10 2 / 3 *
15
```

*Gambar 1. Screenshot Hasil Jawaban Soal 2*

## C. Pembahasan

Pada baris [1], sintaks `#include <iostream>` mengimpor library input/output standar C++ agar `std::cin` dan `std::cout` dapat digunakan untuk membaca dan mencetak data. Pada baris [2], `#include <sstream>` mengimpor library string stream yang menyediakan kelas `std::istringstream`, digunakan untuk memecah sebuah string menjadi token-token kata per kata. Pada baris [3], `#include <string>` mengimpor library string standar C++ agar tipe data `std::string` dan fungsi pendukungnya seperti `std::stoi()` dapat digunakan. Pada baris [4], sintaks `#include "stack.h"` mengimpor file header stack buatan sendiri yang berisi definisi struct `Stack` beserta deklarasi fungsi-fungsi operasi stack seperti `init()`, `push()`, `pop()`, `peek()`.

Pada baris [6], fungsi `main()` didefinisikan sebagai titik awal eksekusi program. Pada baris [7], variabel `n` bertipe `int` dideklarasikan dan langsung diisi dengan nilai dari input standar menggunakan `std::cin >> n`, yang merepresentasikan jumlah elemen atau informasi awal soal. Pada baris [8], `std::cin.ignore()` dipanggil untuk membuang karakter newline `\n` yang tersisa di buffer input setelah pembacaan `n`, sehingga `std::getline()` pada baris berikutnya dapat membaca baris dengan benar tanpa terganggu sisa karakter tersebut.

Pada baris [11-12], variabel `line` bertipe `std::string` dideklarasikan dan diisi dengan satu baris penuh dari input standar menggunakan `std::getline(std::cin, line)`. Baris input ini berisi seluruh ekspresi postfix dalam satu baris yang akan diproses lebih lanjut.

Pada baris [14], objek `iss` bertipe `std::istringstream` dibuat dari string `line`, yang memungkinkan string tersebut diperlakukan seperti stream input sehingga dapat dibaca token per token menggunakan operator `>>`. Pada baris [15-16], variabel `s` bertipe `Stack`

dideklarasikan dan fungsi `init(&s)` dipanggil untuk menginisialisasi stack ke kondisi awal kosong, di mana `&s` adalah alamat memori dari variabel `s` yang dioper sebagai pointer.

Pada baris [18], variabel `token` bertipe `std::string` dideklarasikan untuk menampung setiap token yang dibaca dari `iss`. Pada baris [19], perulangan `while (iss >> token)` membaca token satu per satu dari string stream selama masih ada token yang tersedia.

Pada baris [20], kondisi `if (token == "+" || token == "-" || token == "*" || token == "/")` memeriksa apakah token yang dibaca merupakan salah satu operator aritmatika. Apabila benar, pada baris [21] nilai paling atas stack diambil menggunakan `peek(&s)` dan disimpan ke variabel `b` sebagai operand kedua, kemudian `pop(&s)` menghapusnya dari stack. Pada baris [22], hal yang sama dilakukan untuk mendapatkan operand pertama yang disimpan ke variabel `a`. Urutan pengambilan `b` sebelum `a` penting karena stack bersifat LIFO sehingga operand yang terakhir dimasukkan harus diambil lebih dulu.

Pada baris [25–29], variabel `result` diinisialisasi dengan nilai `0` lalu dihitung sesuai operator yang ditemukan menggunakan serangkaian kondisi `if-else if`. Operator `+` menghasilkan `a + b`, operator `-` menghasilkan `a - b`, operator `*` menghasilkan `a * b`, dan operator `/` menghasilkan `a / b` (pembagian bulat).

Pada baris [31], hasil operasi `result` didorong kembali ke stack menggunakan `push(&s, result)`.

Pada baris [32-33], blok `else` menangani token yang bukan operator, berarti token tersebut adalah bilangan. Fungsi `std::stoi(token)` mengubah string token menjadi nilai integer, lalu `push(&s, ...)` mendorongnya ke atas stack.

Pada baris [37], setelah seluruh token selesai diproses, `peek(&s)` mengambil nilai terakhir yang tersisa di stack sebagai hasil akhir ekspresi postfix, lalu `std::cout` mencetaknya diikuti `std::endl` untuk pindah baris. Program diakhiri dengan `return 0` pada baris [438] yang menandakan eksekusi selesai tanpa error.

# SOAL 3

## Implementasi Queue

Pada file `queue.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `queue.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

### A. Source Code

*Table 4. Source Code queue.h*

```
#ifndef QUEUE_H
#define QUEUE_H

#define MAX 100

struct Queue {
    int data[MAX];
    int* front;
    int* rear;
};

void init(Queue* q);
bool isEmpty(const Queue* q);
bool isFull(const Queue* q);
void enqueue(Queue* q, int value);
void dequeue(Queue* q);
int front(const Queue* q);
int back(const Queue* q);

```

```
#endif
```

*Table 5. Source Code queue.cpp*

```
#include <iostream>
#include "queue.h"

int main() {
    int n, k;
    std::cin >> n >> k;

    int arr[100];
    for (int i = 0; i < n; i++) std::cin >> arr[i];

    if (k <= 0 || k > n) return 0;

    Queue q;
    init(&q);
    int sum = 0;
    bool first = true;

    for (int i = 0; i < k; i++) {
        enqueue(&q, arr[i]);
        sum += arr[i];
    }

    std::cout << sum;

    for (int i = k; i < n; i++) {
        sum -= front(&q);
        dequeue(&q);
        enqueue(&q, arr[i]);
        sum += arr[i];
```

```
30          std::cout << " " << sum;
31      }
32
33      std::cout << std::endl;
34      return 0;
35  }
```

## B. Pembahasan

Pembahasan `queue.h`

Pada baris [1], sintaks `#ifndef QUEUE_H` membuka include guard untuk file `queue.h` guna mencegah pemrosesan ganda oleh kompiler. Pada baris [2], `#define QUEUE_H` mendefinisikan makro pelindung tersebut sehingga file ini aman untuk di-include dari berbagai file sekaligus dalam satu proyek.

Pada baris [4], sintaks `#define MAX 100` mendefinisikan konstanta preprocessor `MAX` bernilai `100` sebagai kapasitas maksimum array penyimpanan data queue. Sama seperti pada `stack.h`, nilai ini akan disubstitusi langsung oleh kompiler sebelum proses kompilasi.

Pada baris [6–10], didefinisikan `struct Queue` yang merepresentasikan struktur data queue berbasis array. Atribut `data[MAX]` bertipe `int` adalah array berkapasitas 100 elemen sebagai tempat penyimpanan data queue. Atribut `front` bertipe `int*` adalah pointer yang menunjuk ke alamat memori elemen paling depan queue, yaitu elemen yang akan dikeluarkan pertama kali saat operasi dequeue dilakukan. Atribut `rear` bertipe `int*` adalah pointer yang menunjuk ke alamat memori elemen paling belakang queue, yaitu posisi di mana elemen baru akan dimasukkan saat operasi enqueue dilakukan.

Pada baris [12], dideklarasikan prototipe fungsi `init()` yang menerima pointer ke `Queue` dan bertugas menginisialisasi queue ke kondisi awal kosong dengan mengatur pointer `front` dan `rear` ke posisi yang sesuai. Pada baris [13], dideklarasikan prototipe fungsi `isEmpty()` yang menerima pointer `const` ke `Queue` dan mengembalikan `bool` untuk memeriksa apakah queue dalam kondisi kosong. Pada baris [14], dideklarasikan prototipe fungsi `isFull()` yang mengembalikan `bool` untuk memeriksa apakah queue sudah mencapai kapasitas maksimum `MAX`. Pada baris [15], dideklarasikan prototipe fungsi

`enqueue()` yang menerima pointer ke `Queue` dan nilai `int` untuk menambahkan elemen baru ke bagian belakang queue. Pada baris [16], dideklarasikan prototipe fungsi `dequeue()` yang menerima pointer ke `Queue` untuk menghapus elemen paling depan dari queue. Pada baris [17], dideklarasikan prototipe fungsi `front()` yang menerima pointer `const` ke `Queue` dan mengembalikan nilai `int` berupa elemen paling depan queue tanpa menghapusnya. Pada baris [18], dideklarasikan prototipe fungsi `back()` yang menerima pointer `const` ke `Queue` dan mengembalikan nilai `int` berupa elemen paling belakang queue tanpa menghapusnya. Pada baris [20], `#endif` menutup blok include guard yang dibuka pada baris pertama.

Pembahasan `queue.cpp`

Pada baris [1], sintaks `#include "queue.h"` mengimpor file header queue buatan sendiri yang berisi definisi struct `Queue` beserta deklarasi fungsi-fungsi operasi queue seperti `init()`, `enqueue()`, `dequeue()`, dan `front()`

Pada baris [2] sintaks `#include <stdexcept>` mengimpor library standar C++ yang menyediakan kelas-kelas exception (pengecualian) bawaan untuk menangani kondisi error saat program berjalan. Library ini digunakan bersama mekanisme `try-catch` untuk menangkap dan melempar error secara terstruktur.

Pada baris [4-6] terdapat Fungsi init

```
void init(Queue* q) {
    q->front = q->data;
    q->rear  = q->data - 1;
```

Pointer front diposisikan di awal array (data), sedangkan rear diposisikan satu slot sebelum front (data - 1). Kondisi rear < front ini menjadi invariant yang menandakan queue kosong, karena secara logis tidak ada rentang elemen yang valid di antara keduanya.

Justifikasi kebenaran: Dengan rear = data - 1, kondisi isEmpty (front > rear) langsung terpenuhi. Ini konsisten karena enqueue pertama akan menaikkan rear menjadi data, sehingga rear == front dan queue berisi tepat satu elemen.

Pada baris [8-9] terdapat Fungsi isEmpty

```
bool isEmpty(const Queue* q) {
    return q->front > q->rear;
```

Queue dikatakan kosong jika pointer front berada di posisi yang lebih tinggi dari rear. Kondisi ini terjadi saat init dilakukan atau saat seluruh elemen telah di-dequeue.

Justifikasi kebenaran: Setiap enqueue menaikkan rear, dan setiap dequeue menaikkan front. Queue kosong ketika front 'menyusul' rear, yaitu front > rear. Ini merupakan invariant yang dipertahankan secara konsisten oleh semua operasi.

Pada baris [13-14] terdapat Fungsi isFull

```
bool isFull(const Queue* q) {
    return q->rear >= q->data + MAX - 1;
```

Queue dikatakan penuh jika pointer rear telah mencapai slot terakhir dari array, yaitu posisi data + MAX - 1.

Justifikasi kebenaran: Array data memiliki kapasitas MAX elemen dengan slot valid dari data[0] hingga data[MAX-1]. Ketika rear menunjuk ke data + MAX - 1, tidak ada slot berikutnya yang dapat digunakan untuk enqueue berikutnya.

Pada baris [17-20] terdapat Fungsi enqueue

```
void enqueue(Queue* q, int value) {
    if (isFull(q))
        throw std::overflow_error("Queue penuh");
    *(++q->rear) = value;
```

Jika queue penuh, exception std::overflow_error dilempar. Jika tidak, pointer rear dinaikkan terlebih dahulu (pre-increment), kemudian nilai disimpan ke lokasi yang ditunjuknya.

Justifikasi kebenaran: Pre-increment pada rear memastikan bahwa rear selalu menunjuk ke elemen valid yang paling baru dimasukkan, analog dengan mekanisme push pada stack. Elemen baru selalu ditempatkan di belakang elemen yang sudah ada, mempertahankan urutan FIFO.

Pada baris [23-26] terdapat Fungsi dequeue

```
void dequeue(Queue* q) {
    if (isEmpty(q))
        throw std::underflow_error("Queue kosong");
    q->front++;
```

Jika queue kosong, exception std::underflow_error dilempar. Jika tidak, pointer front dinaikkan satu posisi, secara efektif 'membuang' elemen terdepan dari perspektif queue.

Justifikasi kebenaran: Karena front selalu menunjuk ke elemen yang paling lama dimasukkan, menaikkan front satu posisi secara semantik mengeluarkan elemen tersebut. Elemen berikutnya yang dimasukkan paling awal kini menjadi elemen terdepan baru.

Pada baris [29-32] terdapat Fungsi front

```
int front(const Queue* q) {
    if (isEmpty(q))
        throw std::underflow_error("Queue kosong");
```

```
    return *(q->front);
```

Jika queue kosong, exception dilempar. Jika tidak, nilai yang ditunjuk oleh pointer front dikembalikan tanpa mengubah state queue.

Justifikasi kebenaran: Pointer front senantiasa menunjuk ke elemen pertama yang dimasukkan dan belum dikeluarkan. Dereferensi langsung menghasilkan nilai yang benar tanpa efek samping.

Pada baris [35-38] terdapat Fungsi back

```
int back(const Queue* q) {
    if (isEmpty(q))
        throw std::underflow_error("Queue kosong");
    return *(q->rear);
```

Jika queue kosong, exception dilempar. Jika tidak, nilai yang ditunjuk oleh pointer rear dikembalikan. Elemen ini adalah elemen yang paling terakhir dimasukkan ke dalam queue.

Justifikasi kebenaran: Pointer rear selalu diperbarui oleh enqueue untuk menunjuk ke elemen terakhir. Dereferensi rear pada kondisi tidak kosong pasti menghasilkan elemen terbelakang yang valid.

Analisis Kompleksitas Queue

| **Fungsi** | **Kompleksitas Waktu** | **Kompleksitas Ruang** |
|---|---|---|
| init | O(1) | O(1) |
| isEmpty | O(1) | O(1) |
| isFull | O(1) | O(1) |
| enqueue | O(1) | O(1) |
| dequeue | O(1) | O(1) |
| front | O(1) | O(1) |
| back | O(1) | O(1) |

Seluruh operasi queue berjalan dalam kompleksitas waktu dan ruang O(1) karena tidak ada perulangan atau alokasi memori dinamis. Semua operasi hanya melibatkan manipulasi pointer tunggal dan akses satu elemen secara langsung. Kompleksitas ruang keseluruhan struktur adalah O(MAX) = O(1) karena kapasitas bersifat konstan.

# SOAL 4

## Si Palui dan Deretan Angka

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah $k$ angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih $k$ angka berurutan.

Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

### Batasan

- $1 \le n \le 100$
- $-1 \le k \le n$
- $-1000 \le A_i \le 1000$

Keterangan:

- $n$: jumlah elemen
- $k$: ukuran jendela
- $A_i$: elemen array

### Format Input

```
n k A1
A2 ... An
```

Keterangan:

- Baris pertama `n` menyatakan jumlah atau panjang array. Lalu `k` besar atau jumlah elemen dalam satu kelompok ayng dibentuk
- Baris kedua berisi simbol `A` sebanyak `n` buah

### Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

**Contoh Kasus**

| Input | Output |
|---|---|
| 5 3<br>1 2 3 4 5 | 6 9 12 |
| 5 5<br>1 2 3 4 5 | 15 |
| 6 2<br>4 -1 2 -3 5 1 | 3 1 -1 2 6 |

## A. Source Code

*Table 6. Source Code prob2.cpp*

```
#include <iostream>
#include "queue.h"

int main() {
    int n, k;
    std::cin >> n >> k;

    int arr[100];
    for (int i = 0; i < n; i++) std::cin >> arr[i];

    if (k <= 0 || k > n) return 0;

    Queue q;
    init(&q);
    int sum = 0;
    bool first = true;

    for (int i = 0; i < k; i++) {
        enqueue(&q, arr[i]);
        sum += arr[i];
```

```
21      }
22
23      std::cout << sum;
24
25      for (int i = k; i < n; i++) {
26          sum -= front(&q);
27          dequeue(&q);
28          enqueue(&q, arr[i]);
29          sum += arr[i];
30          std::cout << " " << sum;
31      }
32
33      std::cout << std::endl;
34      return 0;
35  }
```

## B. Output Program

```
PS C:\KULIAH_TI\SEMESTER 2\Praktikum Algo & Struktur Data\MODUL2\module-2-stack-and-queue-Luckyari21-main> g++ prob2.cpp queue.cpp -o prob
2
PS C:\KULIAH_TI\SEMESTER 2\Praktikum Algo & Struktur Data\MODUL2\module-2-stack-and-queue-Luckyari21-main> ./prob2
5 3
1 2 3 4 5
6 9 12
PS C:\KULIAH_TI\SEMESTER 2\Praktikum Algo & Struktur Data\MODUL2\module-2-stack-and-queue-Luckyari21-main> ./prob2
5 5
1 2 3 4 5
15
PS C:\KULIAH_TI\SEMESTER 2\Praktikum Algo & Struktur Data\MODUL2\module-2-stack-and-queue-Luckyari21-main> ./prob2
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
```

*Gambar 2. Screenshot Hasil Jawaban Soal 4*

## C. Pembahasan

Pada baris [1], sintaks `#include <iostream>` mengimpor library input/output standar C++ sehingga `std::cin` dan `std::cout` dapat digunakan untuk membaca dan mencetak data. Pada baris [2], sintaks `#include "queue.h"` mengimpor file header queue buatan sendiri yang berisi definisi struct `Queue` beserta deklarasi fungsi-fungsi operasi queue seperti `init()`, `enqueue()`, `dequeue()`, dan `front()`.

Pada baris [4], fungsi `main()` didefinisikan sebagai titik awal eksekusi program. Pada baris [5], dua variabel `n` dan `k` bertipe `int` dideklarasikan dan langsung diisi dari input standar menggunakan `std::cin >> n >> k`, di mana `n` adalah banyaknya elemen array dan `k` adalah ukuran jendela (window) untuk perhitungan jumlah bergerak (sliding window sum).

Pada baris [8–9], array `arr` dengan kapasitas 100 elemen bertipe `int` dideklarasikan, kemudian perulangan `for` mengisi setiap elemen array dari input standar menggunakan `std::cin >> arr[i]` sebanyak `n` kali.

Pada baris [11], kondisi `if (k <= 0 || k > n) return 0` berfungsi sebagai validasi input untuk menghentikan program lebih awal apabila nilai `k` tidak valid, yaitu lebih kecil atau sama dengan nol, atau lebih besar dari jumlah elemen `n`.

Pada baris [13–14], variabel `q` bertipe `Queue` dideklarasikan dan fungsi `init(&q)` dipanggil untuk menginisialisasi queue ke kondisi kosong. Pada baris [15], variabel `sum` bertipe `int` diinisialisasi dengan `0` untuk menyimpan jumlah elemen dalam jendela aktif. Pada baris [16], variabel `first` bertipe `bool` diinisialisasi dengan `true`, namun dalam implementasi ini variabelnya tidak digunakan secara aktif dalam logika utama.

Pada baris [18–20], perulangan `for` pertama mengisi queue dengan `k` elemen pertama dari array sekaligus menjumlahkan nilainya ke `sum` menggunakan `enqueue(&q, arr[i])` dan `sum += arr[i]`. Ini merupakan inisialisasi jendela pertama dari algoritma sliding window.

Pada baris [23], `std::cout << sum` mencetak jumlah jendela pertama tanpa diikuti spasi atau newline, karena spasi akan ditambahkan sebelum jumlah berikutnya pada iterasi selanjutnya.

Pada baris [25–30], perulangan `for` kedua menggeser jendela dari posisi `k` hingga akhir array. Pada baris [25], `sum -= front(&q)` mengurangi `sum` dengan elemen paling depan queue yang akan keluar dari jendela. Pada baris [26], `dequeue(&q)` menghapus elemen terdepan dari queue. Pada baris [27], `enqueue(&q, arr[i])` menambahkan elemen baru ke belakang queue sebagai anggota jendela yang baru masuk. Pada baris [28], `sum += arr[i]` menambahkan nilai elemen baru tersebut ke `sum`. Pada baris [29], `std::cout << " " << sum` mencetak spasi diikuti jumlah jendela saat ini sehingga semua output terpisah dengan spasi.

Pada baris [33], `std::cout << std::endl` mencetak newline setelah seluruh hasil sliding window selesai dicetak. Program diakhiri dengan `return 0` pada baris [34] yang menandakan seluruh proses berjalan dengan sukses.

## TAUTAN GIT

https://github.com/DSA26-ULM/module-2-stack-and-queue-Luckyari21